

Pipeline Til Satellitdata

SWSAT Software til Satellit Systemer

Gruppe 9

Navn	Studienummer	Studieretning
Esben Neimann	202204959	CE
Jakob Lund	202007550	CE

Indholdsfortegnelse

Introduktion	2
Tekniske detaljer	2
Arkitektur	2
C4-modellen	3
Niveau 1: Software System	3
Niveau 2: Container	4
Niveau 3: Components	5
Design & Implementation	7
Design	7
Implementation	9
Resultater	11
Ground Station Systems and Scheduling (Lab 4-5)	11
Eksempel	12
Satellite Data Pipelines (Lab 6)	12
Eksempel	12
Earth Observation and EO Data Generation (Lab 7)	13
EO Data Ingestion (Lab 8)	14
Data Catalog (Lab 9)	15
EO Data Visualization (Lab 10)	16
AI/ML into the Data Pipeline (Lab 11)	16
Big Data and Analytics in EO (Lab 12)	17
Diskussion	18
Konklusion	19
Feedback	19
References	19
Bibliografi	19
API-kald og respons	1
Oversigt over kommandoer	1
POST /schedule/run	1
GET /schedule	1
Godkendte passes (20 stk.)	1
Afviste passes (42 stk.)	2
Opsummering	3
GET /schedule/2 (schedule ID = 2)	3
Godkendte passes (20 stk.)	3
Afviste passes (42 stk.)	4
Opsummering	5
GET /rejected-passes/2	5
Dashboard	8

Introduktion

Satellitter udgør i dag en yderst væsentlig rygrad af moderne kommunikation, overvågning og dataindsamling. Dette projekt fører os gennem processen bag udviklingen af et eksemplificeret satellitbaseret system.

Med udgangspunkt i ESA's Sentinel-1 satellit og syntetisk genererede billeder, indsamler vi satellitdata og etablerer en fuld pipeline til databehandling og analyse. Gennem laboratorieøvelser har vi langsomt hen over ugerne opbygget og fuldendt denne pipeline, og rapporten præsenterer derfor vores samlede produkt. Sektionerne gennemgår teknologier, designvalg, arkitekturvalg, implementering samt projektets brug af machine learning. Afslutningsvis diskuteres resultaterne, og rapporten rundes af med en samlet konklusion. Vedhæftet, efter afsnittet Referencer, er bilagene der indeholder de samlede resultater.

Tekniske detaljer

Teknologier:

I denne sektion gennemgår og kategoriserer vi de teknologier, der er anvendt i projektet. Projektet er som helhed skrevet i Python. Det er programmeringssproget vi primært er vant til i forhold til machine learning og matematik. Derudover har det en lang række meget anvendelige biblioteker som vi vurderede var nemmest at arbejde med i denne sammenhæng.

Til lagring af data og flight plans har vi anvendt SQLite (Lab 5 & 6). Det er et bibliotek der er godt integreret med de IDE's vi bruger. Overblik over tabeller er generelt overskueligt og dokumentationen for både SQL som sprog og program er meget tilgængeligt. Vi overvejede kort at anvende Mongo DB da det virkede som en god mulighed for at lære det at kende, men projektets scope gjorde SQLite mest attraktivt.

Inputdata er primært ankommet i JSON format. Dette gælder blandt andet metadata, records, kataloger, flight plans og Earth Observation billeder (EO).

Til syntetisk billedgenerering har vi anvendt NumPy i lab 8 og Pillow i lab 12. Pillow er desuden anvendt som bibliotek til billedbehandling, bl.a til forstærkning af kontraster og edge detection i Lab 12. Numpy har vi begge god erfaring med og pillow er et bibliotek som blev præsenteret af underviseren. Det viste sig at være veldokumenteret og det passede perfekt på de teknikker vi alligevel skulle anvende på de 200 importerede billeder i lab 12.

Inden for frameworks og web har vi primært anvendt Flask til Web dashboardet i lab 11, FastAPI til REST API'et for schedule logik og Uvicorn som server til at køre FastAPI. Flask kender vi fra distribuerede systemer hvor vi også fik undervisning i REST strukturen, så det valgte vi også at gå med i dette projekt. Kommunikationsprotokol er HTTP med GET og POST anvendt ved endpoints.

Arkitektur

Det er vigtigt at bemærke, at arkitekturen progressivt har udviklet sig gennem de forskellige laboratorieøvelser. Vi har bevidst valgt en evolutionær proces, hvor arkitekturen, uge for uge, følger kurset. I takt med at vi har udvidet vores teoretiske og praktiske forståelse, har arkitekturen derfor også ændret sig og knyttet sig til de forskellige laboratorieøvelser.

Vi vil først give en kort oversigt over, hvilke dele af arkitekturen der hører til hvilke øvelser, og derefter give en nærmere beskrivelse af hver blok.

C4-modellen

Først og fremmest baserer vi os på C4 modellen og dens forskellige abstraktionsniveauer. Den er vores ramme for systemets arkitektur hvilket gør det nemt for os at bevæge sig fra et overordnet perspektiv ned mod mere konkrete tekniske detaljer. Ifølge [1] er C4 inddelt i fire niveauer: Software System, Container, Components og Code.

Niveau 1 giver et eksternt overblik over systemet, dets aktører og hvorledes data kontekst flyder. Niveau 2 beskriver de vigtigste containere og deres ansvar, mens niveau 3 fokuserer på udvalgte komponenter, der går igen i projektets designvalg og kode. Niveau 4 behandles primært gennem de relevante code snippets i implementeringsafsnittet.

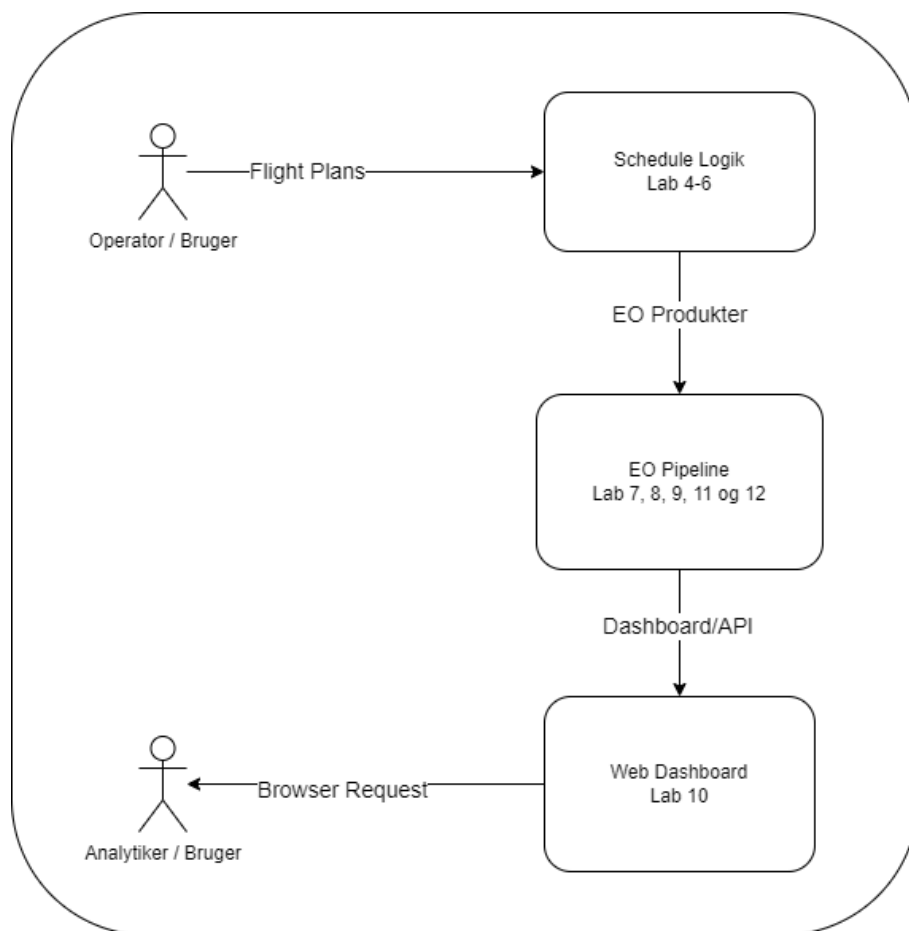
Her beskrives systemets arkitektur gennem diagrammer med tilhørende forklarende billedtekst. Formålet er ikke at dokumentere hver enkelt fil i projektet, men at fremhæve de vigtigste dele af arkitekturen, som har haft størst betydning for systemets opbygning.

Niveau 1: Software System

Det højeste abstraktionsniveau i C4-modellen er softwaresystemet. Dette niveau beskriver det samlede softwaresystem samt de eksterne systemer og aktører, som det relaterer sig til.

I vores tilfælde dækker softwaresystemet over det samlede projekt, vi har arbejdet med gennem kurset. Nedenfor har vi illustreret dette i et System Context diagram [2], som viser systemets overordnede afgrænsning, dets centrale aktører og hvordan de forskellige dele af systemet interagerer med hinanden.

Altså, hvem bruger det, hvad interagerer det med og hvordan flyder data.



Figur 1: System Kontekst over fuld pipeline

Operatøren indsender passes eller flight plans til vores data pipeline. Disse behandles først gennem schedule logikken fra Lab 4–5, som danner grundlag for udvælgelsen af relevante passes. På baggrund af denne logik kan systemet i teorien hente billeder fra de valgte områder og generere Earth Observation PNG billeder. Resultaterne præsenteres efterfølgende i dashboardet, hvor en bruger eller analytiker kan analysere de genererede data.

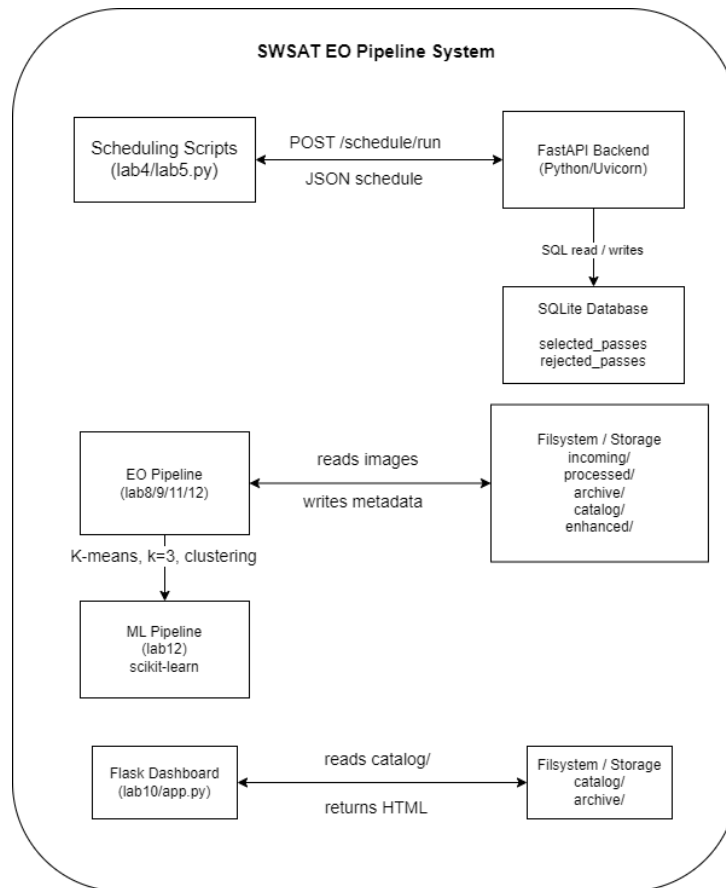
Niveau 2: Container

Dette segment af C4-modellen beskriver de containere, som systemet består af. Et gennemgående tema for vores designvalg er, at vi har forsøgt at holde arkitekturen så simpel som muligt. Dette kommer også til udtryk gennem de forskellige laboratorieøvelser, hvor vi løbende har udvidet systemet og truffet de respektive designvalg.

Ifølge [3], repræsenterer en container en applikation eller en selvstændig del af systemet, som skal være operationel for at resten af det samlede softwaresystem fungerer. En container kan for eksempel være en backend, database, webapplikation eller en pipeline. Herunder har vi lavet en tabel over de vigtigste containers i vores system, den teknologi der anvendes, samt det ansvar den enkelte container står for.

Container	Teknologi	Ansvar
Scheduling Scripts	Python	Indlæser passes, anvender policies og genererer flight plan
FastAPI Backend	Python, FastAPI, Uvicorn	REST API – eksponerer scheduling-data
SQLite Database	SQLite	Lagrer selected/rejected passes pr. schedule-run
EO Pipeline	Python, Pillow, NumPy	States, filflytning, arkivering og katalog
ML Pipeline	Python, scikit-learn	Batch-processing, enhancement og K-means clustering
Flask Dashboard	Python, Flask, Jinja2	Web-UI til browsing af EO-katalog med filtrering
Filsystem/Storage	Disk	Delt lager, hvor alle pipelines læser og skriver data

Derudover har vi illustreret relationerne mellem containerne i diagrammet nedenfor, se Figur 2. Diagrammet viser, hvordan de forskellige dele af systemet hænger sammen, og hvordan data bevæger sig mellem de enkelte containers.



Figur 2: Diagram over containere for niveau 2 i C4-modellen.

Niveau 3: Components

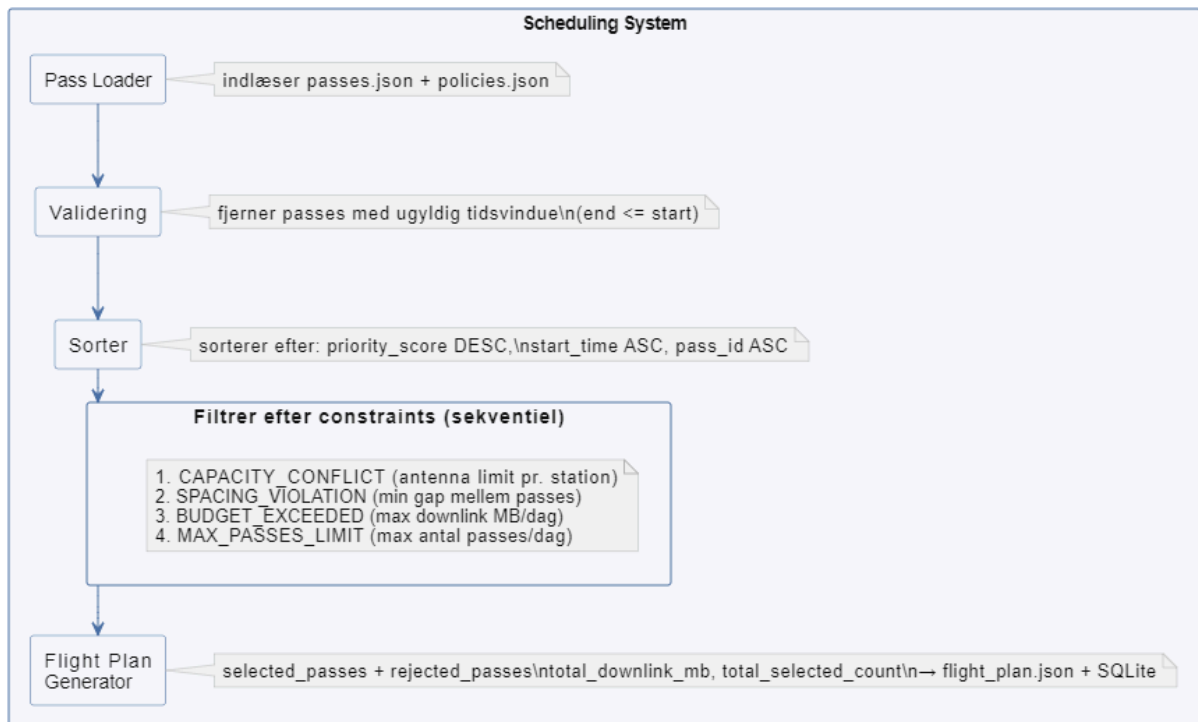
Begrebet komponenter bruges vidt og bredt i softwareverdenen, men i C4-modellen forstås en komponent som en gruppering af relateret funktionalitet, der er skjult bag et veldefineret interface. [4]

Hvor *code* niveauet drejer sig om programmeringssprog og individuelle funktioner, beskriver dette abstraktionsniveau større logiske dele af systemet. En komponent kan derfor bestå af flere klasser, interfaces, moduler eller filer, som tilsammen løser en bestemt opgave i systemet. Herunder beskriver vi de vigtigste komponenter i vores system og deres rolle i den samlede arkitektur. For at undgå en unødigt detaljeret gennemgang af hele systemet, fokuserer vi på de komponenter, der har haft størst betydning for projektet.

Komponenterne vil derfor gå igen i de efterfølgende kapitler, hvor de både knyttes til vores designvalg og til den konkrete implementering. På den måde skabes en rød tråd gennem rapporten, hvor de samme centrale dele af systemet først beskrives arkitektonisk og derefter uddybes gennem kode og tekniske valg.

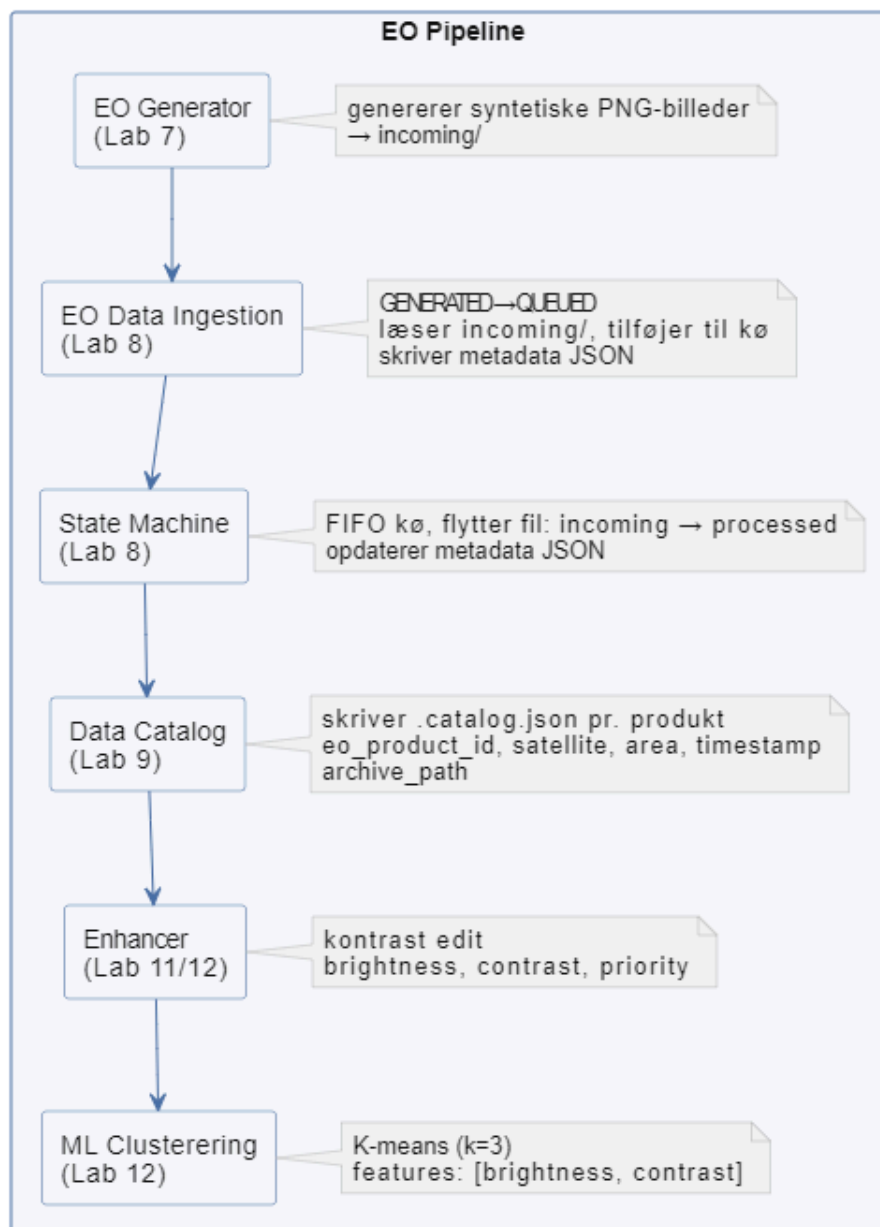
Scheduling:

Første komponent dækker arbejdet fra uge 4-6, nemlig vores komponent der ultimativt generer flight plans.



Figur 3: Scheduling, som viser hvordan en liste med passes bliver hentet ind, valideret, sorteret og gemt.

EO Pipeline: Næste component diagram dækker og giver et overblik over funktionaliteterne fra lab 7-9, 11 & 12.



Figur 4: Pipeline som viser hvordan lab 7-12 hænger sammen i det store billede.

Design & Implementation

For at undgå at beskrive hele implementeringen i detaljer har vi valgt kun at fokusere på de vigtigste designvalg. Afsnittet er opdelt efter de pipelines, vi har arbejdet med i faget. Første del omhandler de valg, vi traf i forbindelse med oprettelsen og behandlingen af flight schedules i Lab 4–6. Anden del fokuserer på Earth Observation-pipelinen, mens tredje del omhandler brugen af machine learning, og hvordan vi opdeler et mere realistisk eksempel i clusters. Det er i denne sidste del, at vi samler den viden, der er opbygget gennem de tidligere laboratorieøvelser.

Design

Lab 4–6: Schedule logik

Et godt eksempel på dette er vores logik vedrørende Scheduling fra Lab 4–6, hvor vi har valgt en sekventiel fremgangsmåde i designet af pipelinen. Schedule logikken er tænkt som en lineær pipeline, hvor flight plans og passes sekventielt kontrolleres ud fra en række restriktioner.

Rækkefølgen i pipelineen er opbygget sådan, at vi først indlæser data, derefter validerer, sorterer og til sidst filtrerer vores flight plans og passes. Hvis en prædefineret regel brydes, afbrydes kæden for det pågældende pass, og evalueres ikke yderligere. På den måde undgår vi unødvendig behandling af data, der allerede er blevet afvist.

Hvert trin i kæden har sin egen egenskab og funktionalitet og følger dermed princippet om single responsibility. Det betyder, at hvert trin har ét klart ansvar i pipelineen, hvilket gør systemet mere overskueligt, lettere at teste og nemmere at udvide.

De fire constraints var givet af læreren, mens sorteringslogikken var vores ansvar.

- CAPACITY_CONFLICT

Er den fysiske hardware grænse for hver ground station

- SPACING_VIOLATION

Sikrer, at der er tilstrækkelig tidsafstand mellem passes på samme ground station.

- BUDGET_EXCEEDED

Sikrer, at den samlede downlink pris ikke overstiger det tilladte budget

- MAX_PASSES_LIMIT

Begrænser antallet af passes, der kan vælges inden for en given kørsel

Sorteringen er primært designet i tre niveauer, hvor det vigtigste kriterium er priority_score fra høj til lav. Derefter sorteres der sekundært efter start_time, hvor det tidligste tidspunkt prioriteres først. Til sidst anvendes pass_id som en deterministisk tiebreaker, hvis to passes ellers har samme prioritet og starttidspunkt.

REST API:

Som en del af kursets første halvdel skulle vi i lab 6 have fokus på at opbygge et API til ovennævnte logik. Når man designer et API, skal der naturligvis træffes en række designvalg.

I designet af API'et har vi anvendt REST som arkitektonisk mønster. API'et består af fire centrale endpoints, der tilsammen håndterer diverse aktiviteter.

Hver kørsel af /schedule/run producerer et nyt ID, hvilket gør det muligt at gemme og sammenligne schedules over tid. På den måde kan tidligere kørsler hentes frem igen og sammenlignes med nyere resultater.

API'et anvender to http metoder, POST og GET. POST anvendes til at udløse oprettelsen af et ny schedule, mens GET anvendes til at hente eksisterende data frem. Der er tre forskellige GET-endpoints: et til at hente det seneste schedule, et til at hente et specifikt schedule baseret på ID og et til at hente afviste passes for et bestemt run af komponenten.

EO Pipeline – Lab 8–9

Vi fortsætter den lineære tilgang i Lab 8 og 9, hvor vi introducerer et nyt design pattern. I øvelsen fra Lab 8 valgte vi at arbejde med en state machine. Hvert Earth Observation produkt indgår i den samme fælles livscyklus, som er defineret af fire forskellige states. Disse states er:

Generated → Queued → Processing → Completed

Indtagelsen af data sker efter FIFO princippet, altså first in, first out, hvor de første batches af 10 produkter, der lægges i køen, også er de første, der bliver behandlet. Produkterne processeres lineært og skifter state afhængigt af, hvor langt de er i processen. På den måde afspejler metadata i JSON

format altid den aktuelle status for hvert produkt. 10 batches af gangen, gør det muligt at behandle flere produkter samlet, men stadig bevare en tydelig og kontrolleret rækkefølge i pipelinen.

Implementation

I dette kapitel vil vi gennemgå de vigtigste dele af koden fra laboratorie øvelserne. Der inkluderes code snippets fra de mest relevante metoder. Ideen er at vi vil afspejle de designvalg vi fortalte om i det tidligere afsnit.

Schedule Pipeline

Denne snippet viser implementeringen af de fire constraints, som vi påfører hvert pass.

```
for current_pass in sorted_passes:
    # rule 1 capacity
    rejection = check_capacity_constraint(current_pass, selected_passes, policies)
    if rejection is not None:
        rejected_passes.append(rejection)
        continue

    # rule 2 spacing
    rejection = check_spacing_constraint(current_pass, selected_passes, policies)
    if rejection is not None:
        rejected_passes.append(rejection)
        continue

    # rule 3 budget
    rejection = check_budget_constraint(current_pass, used_budget, policies)
    if rejection is not None:
        rejected_passes.append(rejection)
        continue

    # rule 4 max passes
    rejection = check_max_passes_constraint(current_pass, selected_passes, policies)
    if rejection is not None:
        rejected_passes.append(rejection)
        continue

    # keep the pass if all rules are ok
    selected_passes.append(current_pass)
    used_budget = used_budget + current_pass["downlink_mb"]
```

REST API

Udviklingen af vores API ved brug af FastAPI kan ses herunder. Snippetten viser vores fire HTTP endpoints, som blev nævnt tidligere.

```
from fastapi import FastAPI
from backend.database.sqlite import (get_latest_schedule,
read_rejectpass_from_schedule_id, initialize_database, get_specific_schedule,
run_schedule)

app = FastAPI()

# POST
@app.post("/schedule/run")
def run_schedule_endpoint():
    run_schedule()
```

```

        return {"message": "schedule ran"}

# GET schedule
@app.get("/schedule")
def read_schedule():
    schedule = get_latest_schedule()
    return {"schedule": schedule}

# GET specific schedule based on id
@app.get("/schedule/{schedule_id}")
def read_specific_schedule(schedule_id: int):
    schedule = get_specific_schedule(schedule_id)
    return {"schedule": schedule}

# get rejected passes based on id
@app.get("/rejected-passes/{schedule_id}")
def get_rejected_passes(schedule_id: int):

    rejected_passes = read_rejectpass_from_schedule_id(schedule_id)
    return {"rejected_passes": rejected_passes}

```

Her ses et eksempel på vores state machine fra Lab 8. I denne del opdateres status for hvert EO produkt, alt efter hvor langt produktet er nået i processen. På den måde kan hvert produkt følges fra det oprettes til det er færdigbehandlet.

State Machine (lab 8) Før produkterne kan processeres i state machine, genereres de syntetiske EO billeder i Lab 7 ved hjælp af NumPy og Pillow med seeded tilfældighed for reproducerbarhed.

Snippetten herunder viser overgangen fra PROCESSING til COMPLETED, hvor filen fysisk flyttes fra incoming/ til processed/.

```

def process_product(product):
    """
    Process a single EO product.
    """

    # Update state to PROCESSING
    product["processing_state"] = "PROCESSING"

    print(f"[WORKER] Processing {product['eo_product_id']}")

    # Simulate processing time
    time.sleep(PROCESSING_DELAY)

    # Move file from incoming/ to processed/
    src = INCOMING_DIR / product["file_name"]
    dst = PROCESSED_DIR / product["file_name"]
    os.rename(src, dst)

    # Update state to COMPLETED
    product["processing_state"] = "COMPLETED"

    print(f"[WORKER] Completed {product['eo_product_id']}")

```

K-Means (lab 12)

Da vi tidligere har haft et længere kursusforløb omkring machine learning, vurderede vi, at det var passende at anvende Scikit-learn's bibliotek til at beregne k-means. Efter flere iterationer, hvor

algoritmen starter med tre tilfældigt valgte centre, konvergerer den mod tre tydelige clusters. Det er vigtigt at bemærke, at k-means kun grupperer data ud fra geometrisk nærhed i feature space.

K-means-algoritmen er relativt simpel og kan beskrives i følgende trin:

1. Vi vælger tre tilfældige punkter, centroids, der repræsenterer centrum for hver cluster.
2. Hvert datapunkt tildeles den nærmeste centroid.
3. Hver centroid flyttes til gennemsnittet af de punkter, der er tildelt den.
4. Trin 2-3 gentages, indtil centroids er konvergeret og clusters er dannet.

k-means er defineret som funktion her, hvor der bliver importeret funktioner fra scikit-learn:

```
def run_clustering(data, k):
    features = np.array([[item["brightness"], item["contrast"]] for item in data])
    kmeans = KMeans(n_clusters=k, random_state=42)
    labels = kmeans.fit_predict(features)

    for i, item in enumerate(data):
        item["cluster"] = f"cluster_{int(labels[i])}"
    return data
```

Den bliver senere kaldt i run_pipeline:

```
def run_pipeline():
    metadata_path = "storage/eo_metadata.json"
    results_path = "storage/eo_results_clustering.json"
    k = 3

    data = load_metadata(metadata_path)
    batch_size = 10
    batches = split_into_batches(data, batch_size)
    all_results = []

    with open("logs/pipeline.log", "w") as log_file:
        for batch_number, batch in enumerate(batches, start=1):
            processed_batch = process_batch(batch, batch_number, log_file)
            all_results.extend(processed_batch)

    all_results = run_clustering(all_results, k)

    with open(results_path, "w") as f:
        json.dump(all_results, f, indent=2)
```

Resultater

Dette afsnit indeholder resultaterne fra vores laboratorieøvelser. Det er inddelt efter uger.

Ground Station Systems and Scheduling (Lab 4-5)

Inputfilen input1_passes_medium.json har 62 passes med felterne pass_id, station_id, start_time, end_time, downlink_mb, og priority_score. Baseret på disse parametre og de fire constraints CAPACITY_CONFLICT, SPACING_VIOLATION, BUDGET_EXCEEDED og MAX_PASSES_LIMIT, blev 20 passes godkendt, mens 42 blev afvist. De godkendte passes blev derefter sorteret først efter priority_score, så efter start_time, og til sidst efter pass_id for at der altid kunne skelnes mellem passes med samme score og starttidspunkt. Afviste passes blev inddelt i kategorier efter afvisningsårsag. Afvisningsårsagen var bare den første constraint, der blev brudt.

Eksempel

Her er et eksempel på hvor pass M60 bliver behandlet i pipeline:

- Pass M60 har station_id GS1, start_time 2026-03-02T00:02:00, end_time 2026-03-02T00:11:00, downlink_mb 450, og priority_score 10.

Først tjekkes det at det er et gyldigt pass ved at start_time ligger før end_time. Det er det, så det går videre til næste step. Nu sorteres alle passes efter priority_score, start_time og pass_id. M60 har den første start_time af alle passes med priority_score 10, så det kommer først i køen.

Det skal nu klare de fire constraints:

- check_capacity_constraint: M60 er det første pass i køen, så GS1s begrænsning på 2 samtidige passes er ikke overskredet.
- check_spacing_constraint: GS1 har en minimum spacing på 3 minutter, og M60 er den første overflyvning, så der er ikke noget at komme for tæt på. Ingen problemer her.
- check_budget_constraint: M60 kræver 450 MB, og det samlede budget for dagen er 12000 MB. Eftersom M60 er det første pass, er der god plads i budgettet.
- check_max_passes_constraint: M60 er det første pass, så det er inden for grænsen på 20 passes.

Eftersom ingen af funktionerne returnerer None, bliver M60 godkendt og tilføjet til listen over godkendte passes.

Satellite Data Pipelines (Lab 6)

API'en bruges til at køre schedule logikken fra lab 4 og 5 over HTTP. Det betyder at den kan køres eksternt og kan give et simpelt interface til eventuelle brugere eller andre systemer, der gerne vil bruge vores schedule logik.

REST metode	Endpoint	Beskrivelse
POST	/schedule/run	Kører schedule og gemmer resultatet i vores database med et nyt ID
GET	/schedule	Henter det seneste schedule med godkendte og afviste passes
GET	/schedule/{schedule_id}	Henter et specifikt schedule baseret på ID
GET	/rejected-passes/{schedule_id}	Henter afviste passes og afvisningsårsager for et givet schedule

Eksempel

Et eksempel på brugen af API'et kunne være følgende:

Kør `curl -X POST http://127.0.0.1:8000/schedule/run` for at køre schedule logikken. Herefter kan `curl http://127.0.0.1:8000/schedule` køres for at hente det seneste schedule, som indeholder både godkendte og afviste passes. For at hente et specifikt schedule køres `curl http://127.0.0.1:8000/schedule/{schedule_id}`.

```
$ curl -X POST http://127.0.0.1:8000/schedule/run
```

```
{"message": "schedule ran"}
```

```
$ curl http://127.0.0.1:8000/schedule
```

```

{
  "schedule": {
    "selected_passes": [
      {"pass_id": "M60", "station_id": "GS1",
       "start_time": "2026-03-02T00:02:00", "end_time": "2026-03-02T00:11:00",
       "downlink_mb": 450.0, "priority_score": 10},
      {"pass_id": "M38", "station_id": "GS1",
       "start_time": "2026-03-02T04:04:00", "end_time": "2026-03-02T04:17:00",
       "downlink_mb": 150.0, "priority_score": 10}
      // ... 18 yderligere passes
    ],
    "rejected_passes": [
      {"pass_id": "M16", "rejection_reason": "CAPACITY_CONFLICT",
       "details": "station_id=GS1, antenna_limit=2, conflicting_passes=M04, M28"},
      {"pass_id": "M14", "rejection_reason": "SPACING_VIOLATION",
       "details": "station_id=GS1, required_spacing_minutes=3,
conflicting_pass_id=M26"},
      {"pass_id": "M07", "rejection_reason": "MAX_PASSES_LIMIT",
       "details": "selected_pass_count=20, max_passes_per_day=20"}
      // ... 39 yderligere passes
    ],
    "total_downlink_mb": 6200.0,
    "total_selected_count": 20
  }
}

```

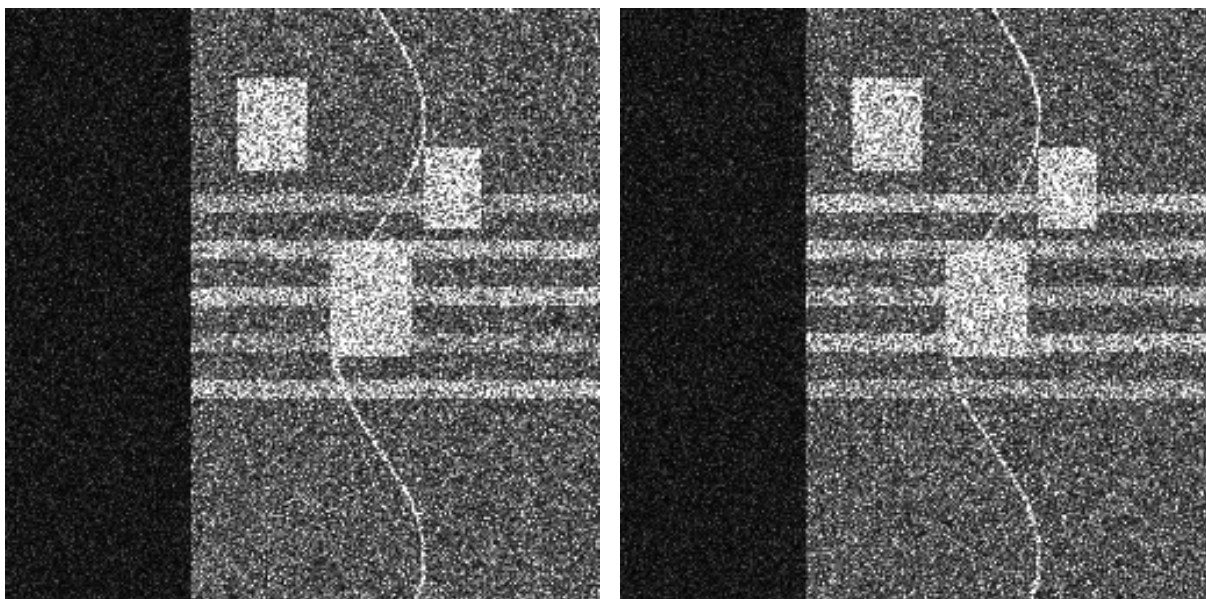
Når den udleverede inputfil `input1_passes_medium.json` køres gennem API'et, bliver 20 passes godkendt og 42 passes afvist. Det ses i de vedlagte bilag.

Earth Observation and EO Data Generation (Lab 7)

Lab 7 handler om at generere syntetiske Earth Observation billeder. Generatoren laver geografiske elementer ved at lægge tilfældige lag oven på hinanden.

- Vand. Mørk venstre side
- Land. Lysere højre side
- Markstriber. Skrå linjer
- Byblokke. Små lyse rektangler
- Vej eller flod. En sinusformet kurve på tværs af landjorden
- Støj. Gamma-distribueret, simulerer SAR-lignende billede-struktur

Her har vi genereret to syntetiske billeder på 256x256 pixels. Dette er deterministisk, da processen er seeded baseret på billedernes ID.



Figur 5: To genererede EO billeder fra Lab 7. De er næsten identiske pga. korrelerede seeds, men de er forskellige.

Når billederne er genereret, bliver de sorteret i mapper baseret på dato. Sammen med hvert billede bliver der genereret tilhørende metadata. Denne metadata indeholder `eo_product_id`, `schedule_id`, `pass_id`, `satellite_id`, `area_name`, `generated_at`, `image_path`, `image_width`, `image_height` og `processing_state`. Denne `processing_state` er hvor lab 8 overtager.

EO Data Ingestion (Lab 8)

Lab 8 fortsætter idéen fra Lab 7 og kører 10 EO-produkter gennem en state machine. Hvert produkt skal igennem følgende fire states før det er færdigbehandlet.

GENERATED → QUEUED → PROCESSING → COMPLETED

Pipelen håndterer alle 10 billeder i en omgang, som bliver lagt i en kø.

Mappe	Ved start	Ved slut
incoming/	10	0
processed/	0	10

Tabel 1: Antal filer i hver mappe før og efter pipelen blev kørt.

Et uddrag af outputtet viser hvordan billederne bliver behandlet over tid.

```
[INGEST] EO-S1A-2026-03-24-001 → QUEUED
...
Queue length: 10

--- START PROCESSING ---

[WORKER] Processing EO-S1A-2026-03-24-001
[WORKER] Completed EO-S1A-2026-03-24-001
Queue length: 9
Visible images: 1
...
Queue length: 0
Visible images: 10
```

Filen med metadata for hvert produkt opdateres hver gang det går videre til næste step, så alle 10 filer i mappen med metadata afslutter med "processing_state": "COMPLETED". Dermed er de færdigbehandlede billeder klar til at blive katalogiseret i lab 9.

Data Catalog (Lab 9)

Lab 9 handler om at katalogisere de færdigbehandlede EO-billeder fra Lab 8. Vi læste billederne ind, organiserede dem i en arkivstruktur og oprettede et katalog over dem.

Arkivstrukturen er som følger

```
archive/  
└─ satellite=Sentinel-1A/  
   │─ area=Aarhus_Harbor/date=2026-03-24/E0-AARHUS-001.png  
   │─ area=Aarhus_University/date=2026-03-24/E0-AARHUS-002.png  
   │─ area=Den_Permanente/date=2026-03-24/E0-AARHUS-003.png  
   └─ ...
```

Dette gør det nemt at finde billeder efter deres metadata som satellit, område og dato. Denne information lægges ind i kataloget som fx

```
{  
  "eo_product_id": "E0-S1A-2026-03-24-001",  
  "satellite_id": "Sentinel-1A",  
  "area_name": "Aarhus Harbor",  
  "timestamp": "2026-03-24T10:25:00Z",  
  "archive_path": "lab9/data/archive/satellite=Sentinel-1A/area=Aarhus_Harbor/  
date=2026-03-24/E0-AARHUS-001.png"  
}
```

I tabellen er der en oversigt over de otte billeder, der blev katalogiseret med deres tilhørende metadata. Stien er blevet forkortet, men de starter alle med lab9/data/archive/satellite=Sentinel-1A/.

EO Product ID	Satellit	Område	Timestamp	Arkiv
EO-S1A-2026-03-24-001	Sentinel-1A	Aarhus Harbor	2026-03-24T10:25:00Z	area=Aarhus_Harbor/ date=2026-03-24/EO-AARHUS-001.png
EO-S1A-2026-03-24-002	Sentinel-1A	Aarhus University	2026-03-24T10:25:00Z	area=Aarhus_University/ date=2026-03-24/EO-AARHUS-002.png
EO-S1A-2026-03-24-003	Sentinel-1A	Den Permanente	2026-03-24T10:25:00Z	area=Den_Permanente/ date=2026-03-24/EO-AARHUS-003.png
EO-S1A-2026-03-24-004	Sentinel-1A	Marselisborg Harbor	2026-03-24T10:25:00Z	area=Marselisborg_Harbor/ date=2026-03-24/EO-AARHUS-004.png
EO-S1A-2026-03-24-005	Sentinel-1A	Hørret	2026-03-24T10:25:00Z	area=Hørret/ date=2026-03-24/EO-AARHUS-005.png
EO-S1A-2026-03-24-006	Sentinel-1A	Ajstrup Strand	2026-03-24T10:25:00Z	area=Ajstrup_Strand/ date=2026-03-24/EO-AARHUS-006.png
EO-S1A-2026-03-24-007	Sentinel-1A	Egå	2026-03-24T10:25:00Z	area=Egå/ date=2026-03-24/EO-AARHUS-007.png
EO-S1A-2026-03-24-008	Sentinel-1A	Aarhus Center	2026-03-24T10:25:00Z	area=Aarhus_Center/ date=2026-03-24/EO-AARHUS-008.png

Tabel 2: De otte billeder i kataloget fra Lab 9.

Dette katalog kan søges i, og her bruges funktionen `query_by_area`, som returnerer alle produkter, der matcher et givent område. Fx returnerer `query_by_area("Aarhus Harbor")` produktet EO-S1A-2026-03-24-001. Kataloget bliver senere brugt som datakilde i dashboardet i Lab 10.

EO Data Visualization (Lab 10)

Lab 10 handler om at bygge et web dashboard til at browse i kataloget fra Lab 9. Dashboardet er bygget i Flask og læser data fra kataloget, så de samme data som bliver genereret i Lab 9 kan ses i dashboardet.

Dashboardet har tre filtre med felterne område, satellit og dato, som kan kombineres frit. Øverst er der også statistikker for hvor mange produkter der er i kataloget, hvor mange der matcher det aktuelle filter, det aktuelt valgte produkt, og seneste timestamp. Klikker man på et produkt, åbner et panel med metadata og et preview af det tilhørende billede, som bliver hentet via `/image/<id>`-endpointet. Derudover er der også et `/api/products`-endpoint, som returnerer alle produkter i kataloget i JSON-format.

Et screenshot af dashboardet er i bilagene Figur 7.

AI/ML into the Data Pipeline (Lab 11)

Lab 11 analyserer kvaliteten af EO-billederne og laver kontrastforstærkede versioner af dem. Der blev kørt fire input-billeder igennem, og for hvert billede blev der beregnet `brightness`, `contrast` og en samlet `quality_score`, som bliver brugt til at finde `priority`. Alle fire blev derefter kontrastforstærket og gemt som nye versioner ved siden af originalen.

ID	Brightness	Contrast	Quality	Priority
EO-002	0.437	0.153	0.295	2
EO-003	0.490	0.063	0.276	2
EO-004	0.321	0.051	0.186	3
EO-005	0.607	0.135	0.371	2

Tabel 3: Kvalitetsmetrikker for de fire input-billeder.

EO-005 har den højeste brightness, contrast og quality_score, og falder derfor i priority 2 ligesom EO-002 og EO-003. Kun EO-004, med en lavere quality_score, ender i priority 3.

Big Data and Analytics in EO (Lab 12)

Lab 12 samler alt fra de tidligere labs i en pipeline. 200 EO-billeder blev kørt igennem i batches af 10 billeder, hvor hvert billede fik en kontrastforstærket version og en quality_score. Bagefter blev de samlet og inddelt i tre clusters med machine learning metoden K-means clustering med k=3 på brightness og contrast.

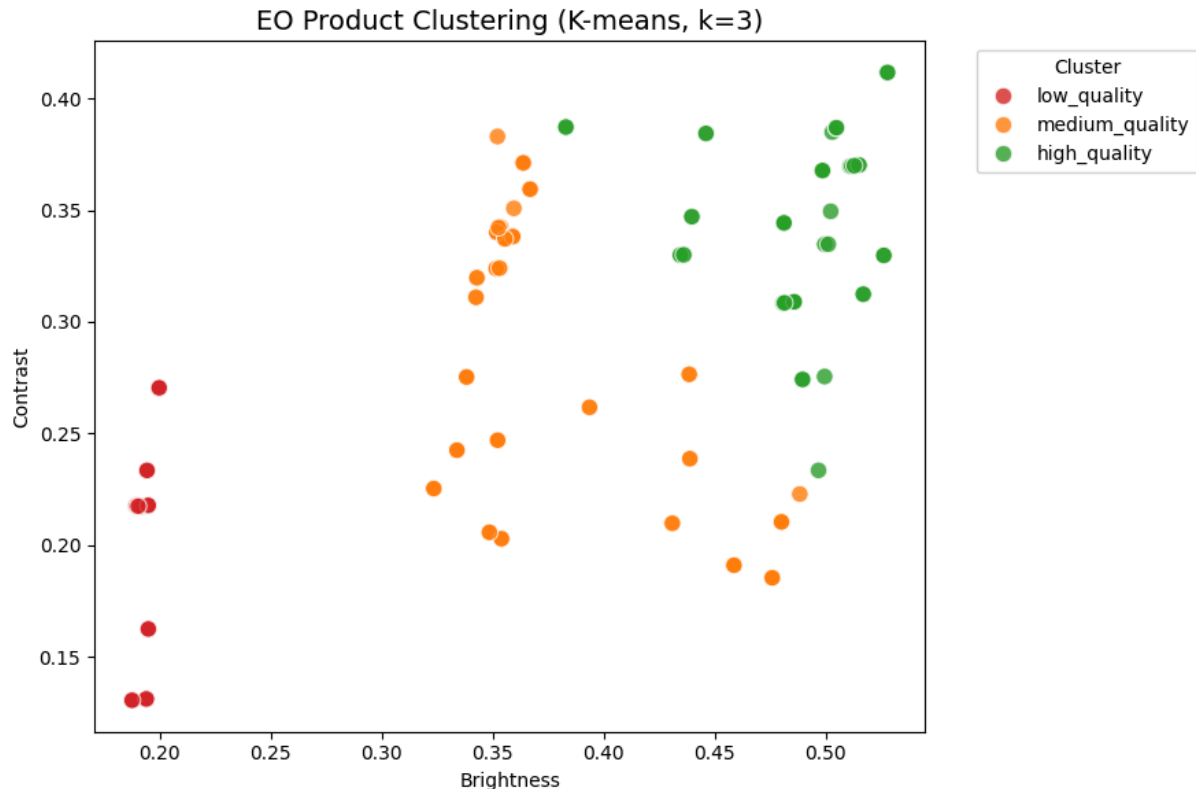
Betydning	Antal	Center
high_quality	54	0.40
medium_quality	69	0.33
low_quality	77	0.19

Tabel 4: Fordeling af de 200 billeder på de tre clusters fra K-means.

Et eksempel på data for et billede efter clustering:

```
{
  "eo_product_id": "EO-001",
  "brightness": 0.195,
  "contrast": 0.218,
  "quality_score": 0.206,
  "priority": 3,
  "cluster": 1,
  "cluster_meaning": "low_quality"
}
```

De samlede clusters vises i Figur 6.



Figur 6: Clustering af produkter. Rød er lav kvalitet, orange er medium kvalitet og grøn er høj kvalitet. Her er lysstyrke (brightness) på x-aksen og kontrast (Contrast) på y-aksen.

Diskussion

Overordnet set vurderer vi at resultaterne viser, at de enkelte laboratorieøvelser ikke blot fungerer isoleret, men kan kobles sammen til en samlet simpel data pipeline. Scheduling delen producerer et godt udgangspunkt for udvælgelse af passes hvor vi kan altid ændre constraints og producerer nye passes.

EO pipeline er vi også ret tilfredse med. Den omsætter til genererede produkter og metadata, mens dashboard og ML delen gør det muligt at analysere og gruppere produkterne. På den måde fungerer projektet som en samlet prototype på et satellitbaseret databehandlingssystem.

På den anden side er systemet stadig en forenklet prototype. Flere dele bygger på syntetiske data, og pipeline er derfor ikke testet mod de samme datamængder, variationer og usikkerheder, som ville opstå ved brug af rigtige billede data fra Sentinel 2. Derfor viser projektet primært, at arkitekturen og arbejdsprocessen fungerer konceptuelt, men ikke nødvendigvis at systemet er helt klar til en egentlig kontekst.

Hvad angår metoder så teknisk har vi fået mere erfaring med C4-modellen, som vi før kun har arbejdet lidt med. C4-modellen gav os en standardiseret måde at beskrive systemet på mens state maskinen i Lab 8, var en simpel måde at holde styr på, hvor i pipeline et produkt befinder sig. Vi har også arbejdet konkret med REST i FastAPI, hvor POST bruges til at udløse handlinger og GET til opslag, og vi har set, hvordan scikit-learn's K-means, relativt simpelt, kan integreres i en eksisterende pipeline.

Et trade-off er at vores pipeline afviser passes. Første gang et pass ikke klarer en constraint, bliver det sorteret fra. Dette betyder, at vi mister informationen, hvis et pass var blevet sorteret fra på flere forskellige constraints. Til gengæld kører pipeline hurtigere, da der er mindre at tjekke. Et

yderligere trade-off er at arkivstrukturen i Lab 9, bruger stier som `satellite=Sentinel-1A/area=Aarhus_Harbor/date=2026-03-24/`. Det gør det nemt at manuelt finde billeder, men det giver også komplekse queries, fx alle billeder med en bestemt `quality_score`, kræver at vi scanner kataloget i stedet for at lave et opslag i en database. Det går dog her fordi datamængden er lille.

Vores løsning har også nogle begrænsninger. Først og fremmest er billederne syntetiske og genereret fra et seed, så `quality_score` og clustering i Lab 11 og Lab 12 siger mere om vores generator end om rigtige satellitbilleder. `Quality_score` er desuden simpel og bygger kun på `brightness` og `contrast`. En rigtig kvalitetsvurdering kunne også tage højde for fx skydække, skarphed og forvrængning. Til sidst har state maskinen i Lab 8 begrænset fejlhåndtering, hvilket betyder, at processen i praksis må genstartes, hvis den crasher. I en fremtidig version ville vi køre på rigtige Sentinel-2 data, udvide `quality_score` med flere features og generelt gøre pipelineen mere robust, så pipelineen kan genoptages efter en fejl.

Det som projektet især har lært os er, at et projekt med en tydelig retning, godt kan laves inkrementelt. Det fungerede rigtig godt i praksis. Vi kendte ikke det endelige system fra start, og det var nemmere at lade C4-modellen vokse fra Software System til komponenter, end at forsøge at designe det hele på en gang. Det betød også, at vi i hvert lab kunne træffe vores egne små valg uden at det havde store konsekvenser for det næste lab.

Konklusion

Vi har bygget en pipeline der dækker hele vejen fra overhead passes til clustrede EO-produkter. Logikken med scheduling fra Lab 4 og Lab 5 godkender 20 ud af 62 passes ud fra de fire constraints og er i Lab 6 lagt bag en API med fire endpoints. Lab 7 til Lab 9 genererer syntetiske 256x256 billeder, kører dem gennem state maskinen `GENERATED` → `QUEUED` → `PROCESSING` → `COMPLETED` og lægger dem ind i et katalog med en arkivstruktur baseret på satellit, område og dato. Lab 10 gør kataloget tilgængeligt i et dashboard, og Lab 11 og Lab 12 udvider med kvalitetsmetrikker og K-means clustering, hvor 200 billeder fordeles på tre clusters, som resulterede i 54 high, 69 medium, og 77 low.

Alle delsystemer kører som tiltænkt, og de enkelte labs hænger sammen i en samlet pipeline. Vi synes at projektet er vellykket, men dog med nogle begrænsninger, nemlig at billederne er syntetiske. Næste skridt ville være at køre pipelineen på rigtige Sentinel-2 data.

Feedback

I starten var det lidt svært at se meningen med de øvelser, som vi lavede. Det blev dog bedre, som vi kom videre i kurset, og til sidst, da vi skulle samle det hele, faldt det på plads. Du kan prøve at lægge mere vægt på, at det hele går op i en større enhed til sidst. Men bare for at lægge vægt på det, så giver det hele god mening, nu hvor vi har samlet det her til sidst.

References

Bibliografi

- [1] S. Brown, "The C4 model for visualising software architecture". 2024.
- [2] S. Brown, "System Context Diagram". 2026.
- [3] S. Brown, "Container". Set: 11. maj 2026. [Online]. Tilgængelig hos: <https://c4model.com/abstractions/container>
- [4] S. Brown, "Component". Set: 11. maj 2026. [Online]. Tilgængelig hos: <https://c4model.com/abstractions/component>

API-kald og respons

Oversigt over kommandoer

#	Kommando	Resultat
1	POST /schedule/run	Nyt schedule oprettet
2	GET /schedule	Henter seneste schedule
3	GET /schedule/2	Henter schedule med ID = 2
4	GET /rejected-passes/2	Henter afviste passes for schedule ID = 2

POST /schedule/run

```
$ curl -X POST http://127.0.0.1:8000/schedule/run  
{  
  "message": "schedule ran"  
}
```

GET /schedule

```
$ curl http://127.0.0.1:8000/schedule
```

Godkendte passes (20 stk.)

Alle datoer er 2026-03-02.

ID	Pass	Station	Start	Slut	MB	Prioritet
41	M60	GS1	00:02	00:11	450	10
42	M38	GS1	04:04	04:17	150	10
43	M56	GS3	16:20	16:28	150	10
44	M21	GS1	18:35	18:51	350	10
45	M58	GS3	20:05	20:15	200	10
46	M59	GS1	22:15	22:29	200	10
47	M25	GS2	02:29	02:45	350	9
48	M04	GS1	08:14	08:30	150	9
49	M28	GS1	08:16	08:32	250	9
50	M09	GS2	18:16	18:24	500	9
51	M61	GS1	06:10	06:24	300	8
52	M62	GS1	06:10	06:22	250	8
53	M05	GS1	10:34	10:48	300	8
54	M24	GS1	00:10	00:24	200	7
55	M53	GS2	10:16	10:31	350	7
56	M06	GS3	12:17	12:25	250	7
57	M19	GS3	14:20	14:31	500	7
58	M36	GS3	00:12	00:24	450	6
59	M13	GS1	02:24	02:36	500	6
60	M26	GS1	04:07	04:23	350	6

Afviste passes (42 stk.)

ID	Pass	Årsag	Detaljer
85	M16	CAPACITY_CONFLICT	station_id=GS1, antenna_limit=2, conflicting_passes=M04, M28
86	M34	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M58
87	M27	CAPACITY_CONFLICT	station_id=GS1, antenna_limit=2, conflicting_passes=M61, M62
88	M43	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M19
89	M07	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
90	M08	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
91	M44	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
92	M14	SPACING_VIOLATION	station_id=GS1, required_spacing_minutes=3, conflicting_pass_id=M26, actual_gap_minutes=0.0
93	M41	CAPACITY_CONFLICT	station_id=GS2, antenna_limit=1, conflicting_passes=M53
94	M10	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
95	M48	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
96	M12	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M36
97	M01	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
98	M37	CAPACITY_CONFLICT	station_id=GS2, antenna_limit=1, conflicting_passes=M25
99	M50	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
100	M15	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
101	M51	SPACING_VIOLATION	station_id=GS1, required_spacing_minutes=3, conflicting_pass_id=M61, actual_gap_minutes=2.0
102	M18	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
103	M54	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
104	M32	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
105	M46	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
106	M35	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
107	M29	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
108	M42	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
109	M20	SPACING_VIOLATION	station_id=GS3, required_spacing_minutes=4, conflicting_pass_id=M56, actual_gap_minutes=1.0
110	M22	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
111	M47	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
112	M02	CAPACITY_CONFLICT	station_id=GS1, antenna_limit=2, conflicting_passes=M38, M26
113	M39	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
114	M03	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20

ID	Pass	Årsag	Detaljer
115	M31	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M19
116	M33	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
117	M45	SPACING_VIOLATION	station_id=GS2, required_spacing_minutes=5, conflicting_pass_id=M09, actual_gap_minutes=2.0
118	M23	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
119	M49	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
120	M40	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
121	M52	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
122	M17	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
123	M30	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
124	M55	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
125	M57	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
126	M11	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20

Opsummering

total_downlink_mb	6200.0
total_selected_count	20

GET /schedule/2 (schedule ID = 2)

\$ curl http://127.0.0.1:8000/schedule/2

Godkendte passes (20 stk.)

ID	Pass	Station	Start	Slut	MB	Prioritet
21	M60	GS1	00:02	00:11	450	10
22	M38	GS1	04:04	04:17	150	10
23	M56	GS3	16:20	16:28	150	10
24	M21	GS1	18:35	18:51	350	10
25	M58	GS3	20:05	20:15	200	10
26	M59	GS1	22:15	22:29	200	10
27	M25	GS2	02:29	02:45	350	9
28	M04	GS1	08:14	08:30	150	9
29	M28	GS1	08:16	08:32	250	9
30	M09	GS2	18:16	18:24	500	9
31	M61	GS1	06:10	06:24	300	8
32	M62	GS1	06:10	06:22	250	8
33	M05	GS1	10:34	10:48	300	8
34	M24	GS1	00:10	00:24	200	7

ID	Pass	Station	Start	Slut	MB	Prioritet
35	M53	GS2	10:16	10:31	350	7
36	M06	GS3	12:17	12:25	250	7
37	M19	GS3	14:20	14:31	500	7
38	M36	GS3	00:12	00:24	450	6
39	M13	GS1	02:24	02:36	500	6
40	M26	GS1	04:07	04:23	350	6

Afviste passes (42 stk.)

ID	Pass	Årsag	Detaljer
43	M16	CAPACITY_CONFLICT	station_id=GS1, antenna_limit=2, conflicting_passes=M04, M28
44	M34	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M58
45	M27	CAPACITY_CONFLICT	station_id=GS1, antenna_limit=2, conflicting_passes=M61, M62
46	M43	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M19
47	M07	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
48	M08	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
49	M44	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
50	M14	SPACING_VIOLATION	station_id=GS1, required_spacing_minutes=3, conflicting_pass_id=M26, actual_gap_minutes=0.0
51	M41	CAPACITY_CONFLICT	station_id=GS2, antenna_limit=1, conflicting_passes=M53
52	M10	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
53	M48	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
54	M12	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M36
55	M01	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
56	M37	CAPACITY_CONFLICT	station_id=GS2, antenna_limit=1, conflicting_passes=M25
57	M50	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
58	M15	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
59	M51	SPACING_VIOLATION	station_id=GS1, required_spacing_minutes=3, conflicting_pass_id=M61, actual_gap_minutes=2.0
60	M18	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
61	M54	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
62	M32	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
63	M46	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
64	M35	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
65	M29	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20

ID	Pass	Årsag	Detaljer
66	M42	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
67	M20	SPACING_VIOLATION	station_id=GS3, required_spacing_minutes=4, conflicting_pass_id=M56, actual_gap_minutes=1.0
68	M22	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
69	M47	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
70	M02	CAPACITY_CONFLICT	station_id=GS1, antenna_limit=2, conflicting_passes=M38, M26
71	M39	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
72	M03	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
73	M31	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M19
74	M33	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
75	M45	SPACING_VIOLATION	station_id=GS2, required_spacing_minutes=5, conflicting_pass_id=M09, actual_gap_minutes=2.0
76	M23	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
77	M49	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
78	M40	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
79	M52	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
80	M17	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
81	M30	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
82	M55	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
83	M57	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
84	M11	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20

Opsummering

total_downlink_mb	6200.0
total_selected_count	20

GET /rejected-passes/2

\$ curl http://127.0.0.1:8000/rejected-passes/2

Returnerer kun rejected_passes-listen for schedule ID = 2 – samme indhold som tabel A.4.2.

ID	Pass	Årsag	Detaljer
43	M16	CAPACITY_CONFLICT	station_id=GS1, antenna_limit=2, conflicting_passes=M04, M28
44	M34	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M58
45	M27	CAPACITY_CONFLICT	station_id=GS1, antenna_limit=2, conflicting_passes=M61, M62
46	M43	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M19

ID	Pass	Årsag	Detaljer
47	M07	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
48	M08	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
49	M44	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
50	M14	SPACING_VIOLATION	station_id=GS1, required_spacing_minutes=3, conflicting_pass_id=M26, actual_gap_minutes=0.0
51	M41	CAPACITY_CONFLICT	station_id=GS2, antenna_limit=1, conflicting_passes=M53
52	M10	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
53	M48	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
54	M12	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M36
55	M01	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
56	M37	CAPACITY_CONFLICT	station_id=GS2, antenna_limit=1, conflicting_passes=M25
57	M50	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
58	M15	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
59	M51	SPACING_VIOLATION	station_id=GS1, required_spacing_minutes=3, conflicting_pass_id=M61, actual_gap_minutes=2.0
60	M18	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
61	M54	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
62	M32	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
63	M46	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
64	M35	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
65	M29	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
66	M42	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
67	M20	SPACING_VIOLATION	station_id=GS3, required_spacing_minutes=4, conflicting_pass_id=M56, actual_gap_minutes=1.0
68	M22	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
69	M47	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
70	M02	CAPACITY_CONFLICT	station_id=GS1, antenna_limit=2, conflicting_passes=M38, M26
71	M39	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
72	M03	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
73	M31	CAPACITY_CONFLICT	station_id=GS3, antenna_limit=1, conflicting_passes=M19
74	M33	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
75	M45	SPACING_VIOLATION	station_id=GS2, required_spacing_minutes=5, conflicting_pass_id=M09, actual_gap_minutes=2.0
76	M23	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
77	M49	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20

ID	Pass	Årsag	Detaljer
78	M40	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
79	M52	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
80	M17	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
81	M30	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
82	M55	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
83	M57	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20
84	M11	MAX_PASSES_LIMIT	selected_pass_count=20, max_passes_per_day=20

Dashboard

← → ↻ http://127.0.0.1:5000

EO Dashboard

[Dashboard](#) [View API](#) [Open EO Browser](#)

Area: Satellite: Date: Apply Filters

System Status

Total products: 8

Filtered products: 8

Selected product: EO-S1A-2026-03-24-001

Latest timestamp: 2026-03-24T10:25:00Z

EO Product List

[EO-S1A-2026-03-24-001](#) | [Aarhus Harbor](#) | [2026-03-24T10:25:00Z](#)
[EO-S1A-2026-03-24-002](#) | [Aarhus University](#) | [2026-03-24T10:25:00Z](#)
[EO-S1A-2026-03-24-003](#) | [Den Permanente](#) | [2026-03-24T10:25:00Z](#)
[EO-S1A-2026-03-24-004](#) | [Marselisborg Harbor](#) | [2026-03-24T10:25:00Z](#)
[EO-S1A-2026-03-24-005](#) | [Hørret](#) | [2026-03-24T10:25:00Z](#)
[EO-S1A-2026-03-24-006](#) | [Ajstrup Strand](#) | [2026-03-24T10:25:00Z](#)
[EO-S1A-2026-03-24-007](#) | [Egå](#) | [2026-03-24T10:25:00Z](#)
[EO-S1A-2026-03-24-008](#) | [Aarhus Center](#) | [2026-03-24T10:25:00Z](#)

Selected Product

ID: EO-S1A-2026-03-24-001

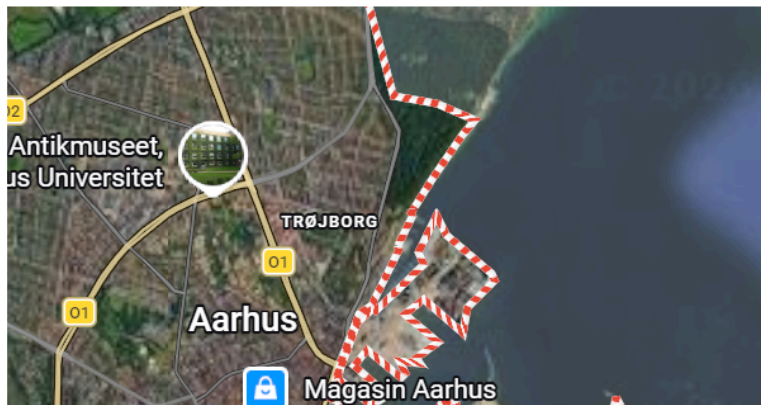
Satellite: Sentinel-1A

Area: Aarhus Harbor

Timestamp: 2026-03-24T10:25:00Z

Archive Path: lab9/data/archive/satellite=Sentinel-1A/area=Aarhus_Harbor/date=2026-03-24/EO-AARHUS-001.png

Image Preview



Figur 7: Billede af online dashboard.